

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

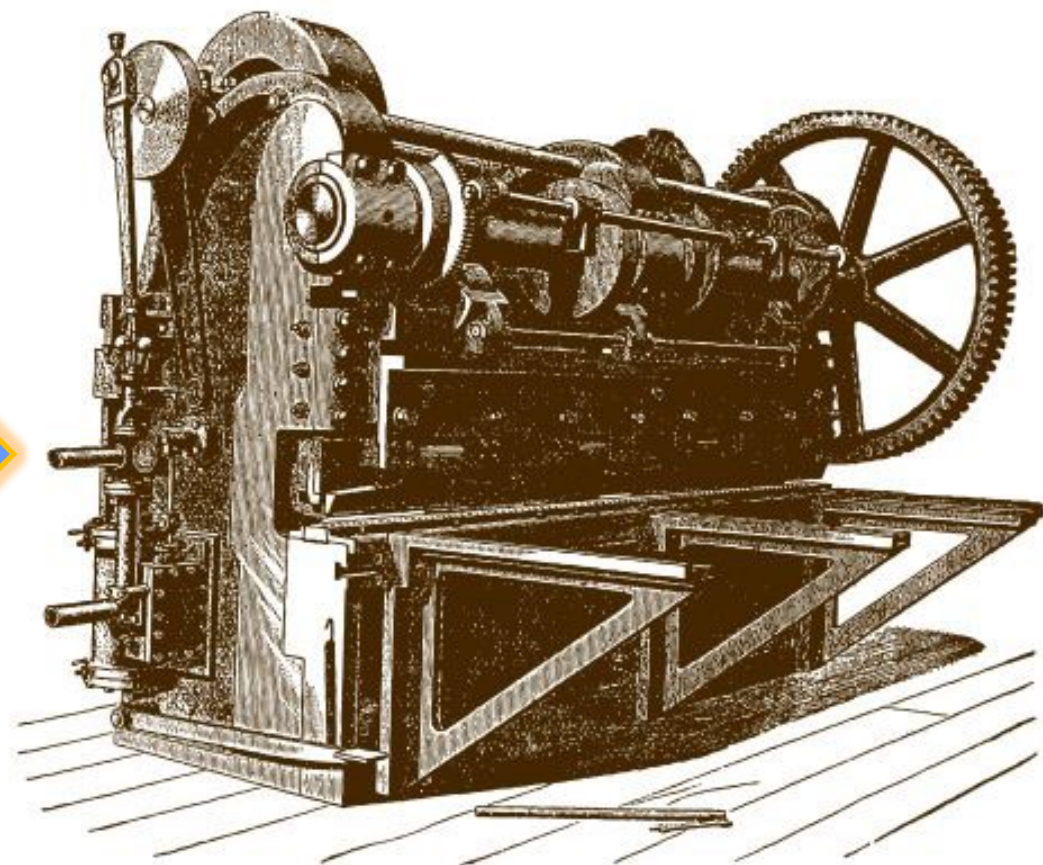
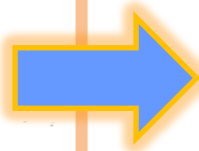
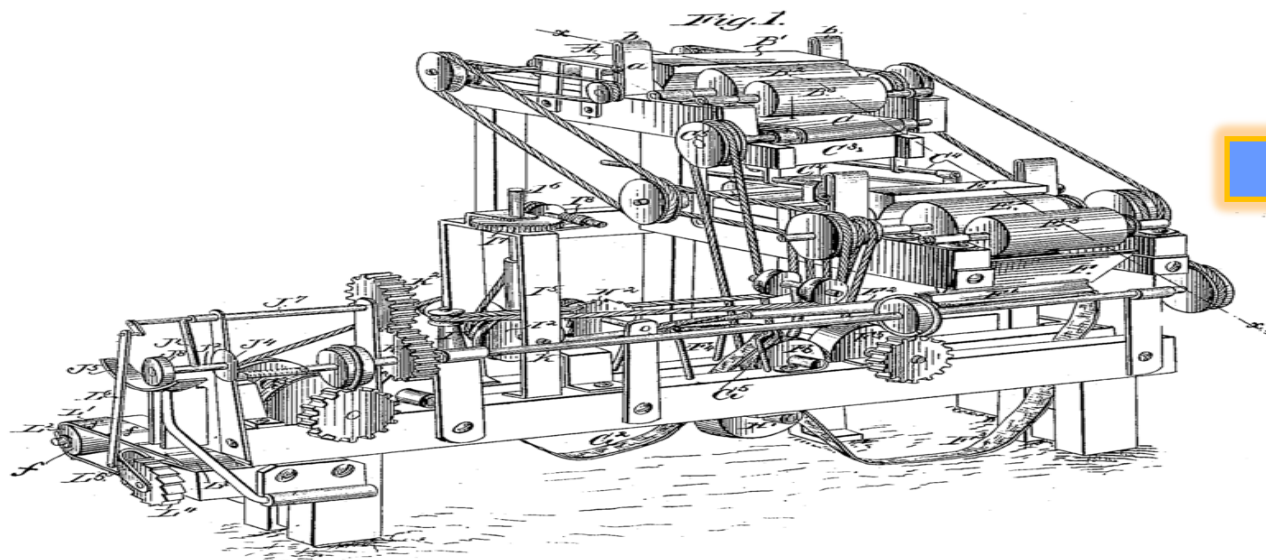
Prof. Dr. Stephan Diehl
Informatik, Universität Trier



Implementierung einer TRAM

TRAM

Trierer Abstrakte Maschine



Assembler

- Assemblersprache
 - Textuelle Darstellung des Maschinencodes
 - Mit Kommentaren
 - Label = symbolische Adressen
- Assembler
 - Wandelt die textuelle Darstellung in die interne Darstellung der (abstrakten) Maschine um
 - Symbolische Adressen (Label) werden durch absolute Adressen ersetzt

TRAM Assemblersprache

```
// Quellkode:  $y = x*3+5*2$   
// Annahmen: Variable x durch Kellerzelle 0  
// und Variable y durch Kellerzelle 1 implementiert,  
// sowie PP=0, FP=0 und TOP=1.  
CONST 6  
STORE 0 0  
LOAD 0 0  
CONST 3  
MUL  
CONST 5  
CONST 2  
MUL  
ADD  
STORE 1 0  
HALT
```

```
# Alternatives Kommentarzeichen  
# Spring hin und her  
GOTO MAIN  
FUN,ALIAS: CONST 42  
RETURN  
MAIN: CONST 5  
L1: CONST 1  
    SUB  
    IFZERO L2  
    GOTO L1  
L2: CONST 1  
    CONST 2  
    INVOKE 2 FUN 0  
    INVOKE 2 ALIAS 0  
    ADD  
    HALT
```

Label → Adressen

Alternatives Kommentarzeichen

Spring hin und her

GOTO **MAIN**

FUN,ALIAS: CONST 42

RETURN

MAIN: CONST 5

L1: CONST 1

SUB

IFZERO **L2**

GOTO **L1**

L2: CONST 1

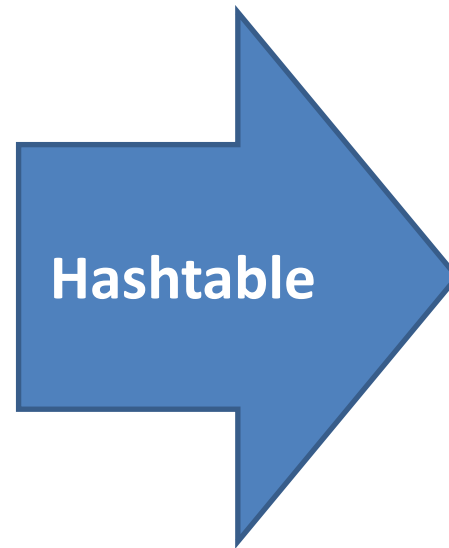
CONST 2

INVOKE 2 **FUN** 0

INVOKE 2 **ALIAS** 0

ADD

HALT



Alternatives Kommentarzeichen

Spring hin und her

0: GOTO **3**

1: CONST 42

2: RETURN

3: CONST 5

4: CONST 1

5: SUB

6: IFZERO **8**

7: GOTO **4**

8: CONST 1

9: CONST 2

10: INVOKE 2 **1** 0

11: INVOKE 2 **1** 0

12: ADD

13: HALT

1. Durchlauf:
Label Adresse zuweisen
2. Durchlauf:
Label in Argumenten ersetzen

Label finden und in Hashmap eintragen

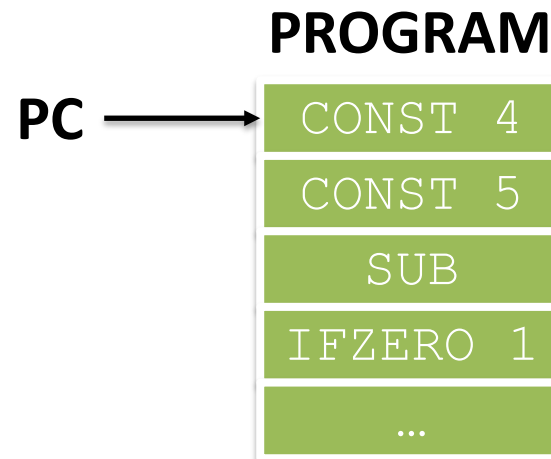
```
while((line = in.readLine())!=null)
{
    line=line.trim();
    if (line.startsWith("//") || line.startsWith("#") ) continue;
    if (line.contains(":")) {
        String[] labels=line.substring(0, line.indexOf(':')).split(",");
        for (String label : labels)
            labelmap.put(label, lineNumber);
        line = line.substring(line.indexOf(':')+1).trim();
    }
    lines.add(line);
    lineNumber++;
}
```

TRAM in Java

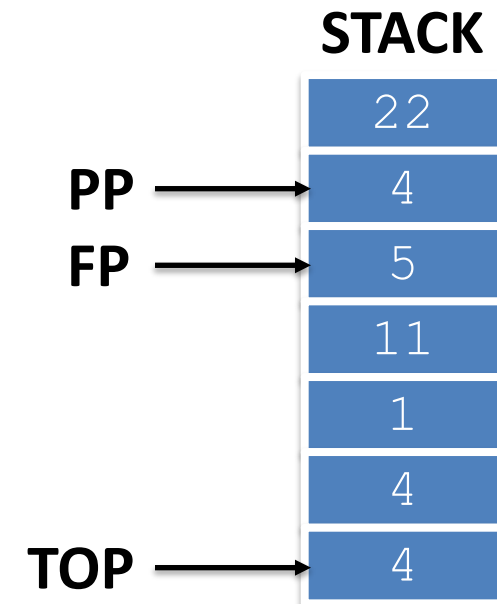
- Register:

- **PC** : zeigt auf aktuell auszuführende Instruktion (program counter)
- **PP**: zeigt auf das erste Argument (parameter pointer)
- **FP**: zeigt auf den Beginn des Kellerrahmens (frame pointer)
- **TOP**: zeigt auf die oberste, belegte Kellerzelle

```
while (PC >= 0)
    execute(program[PC]);
```



Objekte der Klasse
Instruction



int oder Objekte
der Klasse **Integer**

Erweiterte Maschinenbefehle

Ermöglichen Zugriff auf Parameter der statischen Vorgänger:

store k d : $\text{STACK}[\text{spp}(d, PP, FP) + k] = \text{STACK}[\text{TOP}]$
 $\text{TOP} = \text{TOP} - 1$
 $\text{PC} = \text{PC} + 1$

load k d : $\text{STACK}[\text{TOP} + 1] = \text{STACK}[\text{spp}(d, PP, FP) + k]$
 $\text{TOP} = \text{TOP} + 1$
 $\text{PC} = \text{PC} + 1$

Die erweiterten Maschinenbefehle ersetzen `store k` und `load k`

Erstes Projekt

- Implementierung der TRAM in Java
- Debug-Modus (log4j, nach stdout und in Datei)
- TRAM-Code für rekursiven Euklidischen Algorithmus für ggT

<https://betterstack.com/community/guides/logging/how-to-start-logging-with-log4j/>